

NC44 — Terminales de pago en POS: cómo funcionan hoy y propuesta de cambio

1. Resumen del requerimiento

El cliente NC44 quiere **un único método de pago "Tarjeta"** que pueda usar una terminal de pago distinta en cada caja, sin tener que crear un método de pago por cada terminal/caja.

Las "VeriFone" del requerimiento son, en este repo, los módulos que vinculan las terminales físicas de pago al POS:

- **pos_azul** — terminal **Ingenico Lane/7000** (conexión HTTPS directa a la IP del terminal).
- **pos_cardnet** — **Pin Pad CardNET** (conexión vía servicio puente .NET "Bridge" que a su vez habla con la IP del Pin Pad).

Hoy Odoo no permite el escenario "un método, varias terminales" porque **la terminal se configura dentro del método de pago** (`pos.payment.method`), no dentro de la caja (`pos.config`).

2. Cómo funciona HOY (código real del repo)

En ambos módulos, los datos de conexión de la terminal viven como campos del modelo `pos.payment.method` . La pantalla de pago del POS lee esos datos **desde el método de pago**, no desde la caja.

Dónde está la terminal hoy

pos_azul — `pos_azul/models/pos_payment_method.py` :

```
ingenico_terminal_ip    = fields.Char(...)    # IP del terminal Ingenico
ingenico_terminal_port  = fields.Char(...)    # puerto (default 9000)
ingenico_timeout        = fields.Integer(default=40)
```

pos_cardnet — pos_cardnet/models/pos_payment_method.py :

```
cardnet_bridge_url     = fields.Char(...)    # URL del servicio Bridge .NET
cardnet_api_key        = fields.Char(...)
cardnet_local_ip       = fields.Char(...)    # IP del PC que corre el Bridge
cardnet_remote_ip      = fields.Char(...)    # IP del Pin Pad
cardnet_remote_port    = fields.Integer(default=7060)
cardnet_timeout        = fields.Integer(default=90)
# + cardnet_print_mode, cardnet_enable_installments, cardnet_auto_close_batch
```

Cómo llegan al frontend

pos.session._loader_params_pos_payment_method añade esos campos a la carga del POS (en pos_azul/models/pos_session.py y pos_cardnet/models/pos_session.py).

Cómo los lee la pantalla de pago

En el `setup()` del integrador, la IP se lee **del método de pago**:

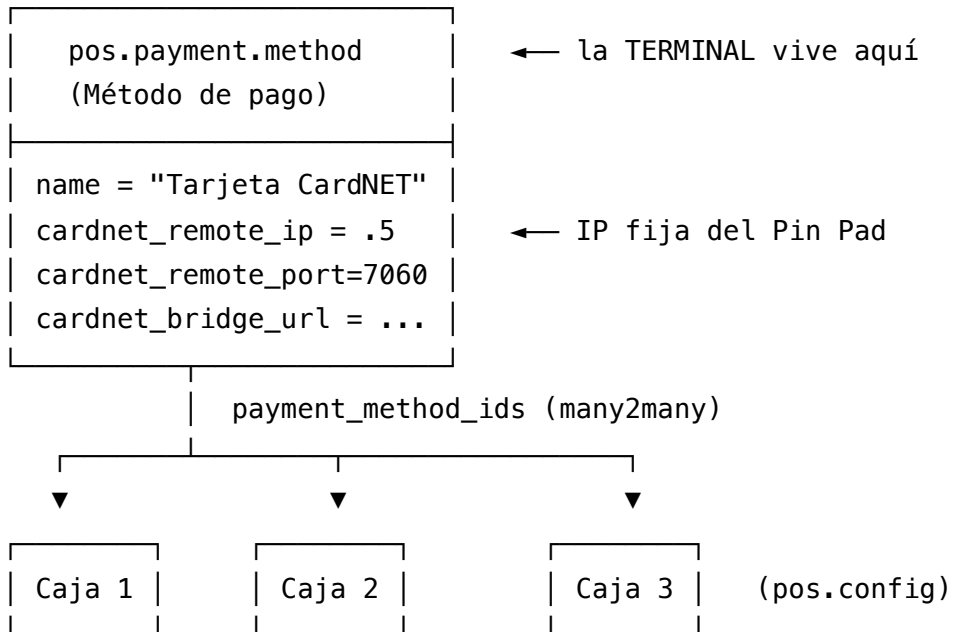
pos_azul/static/src/app/payment_ingenico.js :

```
const baseUrl = this.payment_method.ingenico_terminal_port
  ? `https://${this.payment_method.ingenico_terminal_ip}:${this.payment_method.ingeni
  : `https://${this.payment_method.ingenico_terminal_ip}`;
```

pos_cardnet/static/src/app/payment_ingenico.js :

```
baseUrl:    this.payment_method.cardnet_bridge_url,
localIp:    this.payment_method.cardnet_local_ip,
remoteIp:   this.payment_method.cardnet_remote_ip,
remotePort: this.payment_method.cardnet_remote_port || 7060,
```

Modelo de datos actual



Como la IP está **en el método**, todas las cajas que comparten ese método apuntan a la **misma** terminal física.

Consecuencia: 1 método de pago por terminal

Para que cada caja hable con su propia terminal, hoy hay que clonar el método:

Tienda A

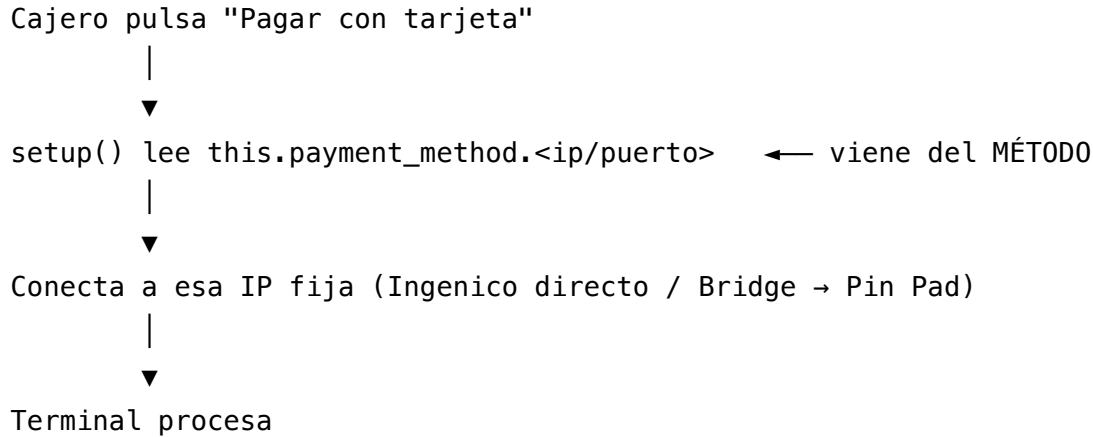
├─ Caja 1 → Método "CardNET Caja 1" → Pin Pad 10.0.0.5
├─ Caja 2 → Método "CardNET Caja 2" → Pin Pad 10.0.0.6
└─ Caja 3 → Método "CardNET Caja 3" → Pin Pad 10.0.0.7

Tienda B

├─ Caja 1 → Método "CardNET B-Caja 1" → Pin Pad 10.1.0.5
└─ Caja 2 → Método "CardNET B-Caja 2" → Pin Pad 10.1.0.6

Problema: N cajas = N métodos de pago (por cada integrador: Ingenico y CardNET). Difícil de mantener, reportes de ventas por método fragmentados, alta complejidad operativa.

Flujo de pago actual



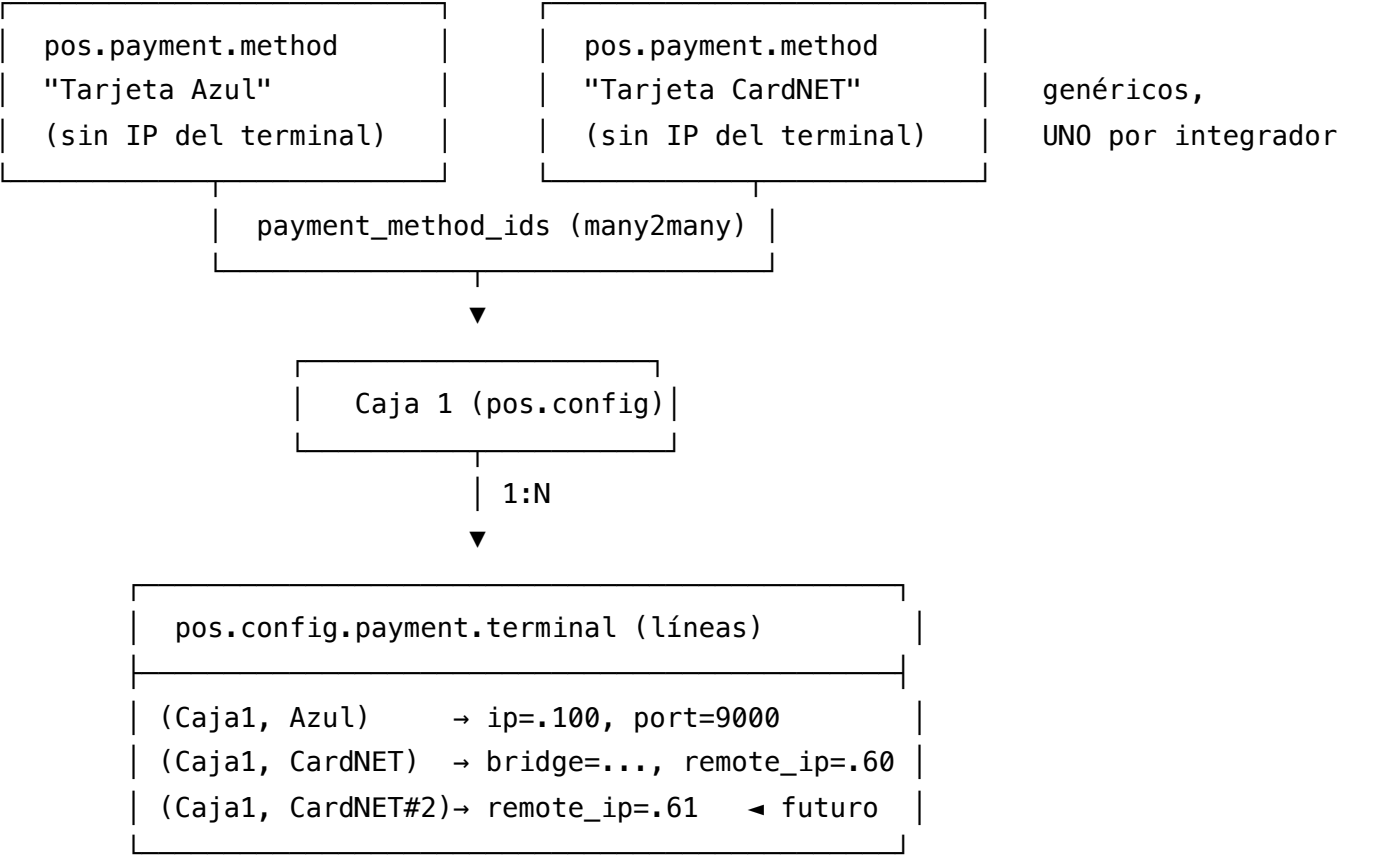
3. Propuesta: terminal por CAJA (pos.config)

Mover los datos de conexión de la terminal **desde el método de pago hacia la caja**. El método de pago queda genérico ("Tarjeta CardNET", "Tarjeta Ingenico") y la caja define a qué terminal física apunta cada método.

Como una caja puede tener **varios métodos a la vez** (azul + cardnet, e incluso 2 terminales del mismo integrador), la conexión NO se guarda como campos planos en `pos.config`, sino en un **modelo intermedio por línea**:

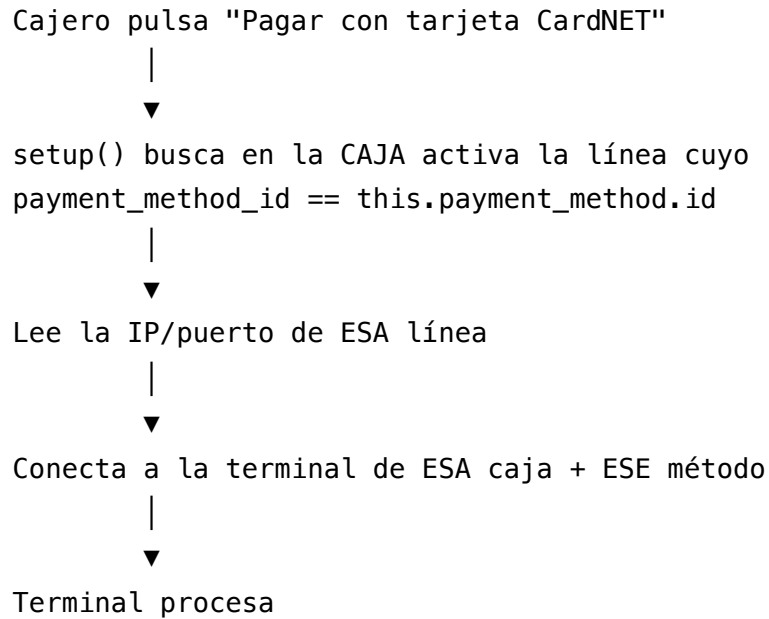
`pos.config.payment.terminal`, una fila por **(caja, método de pago)**.

Modelo de datos propuesto

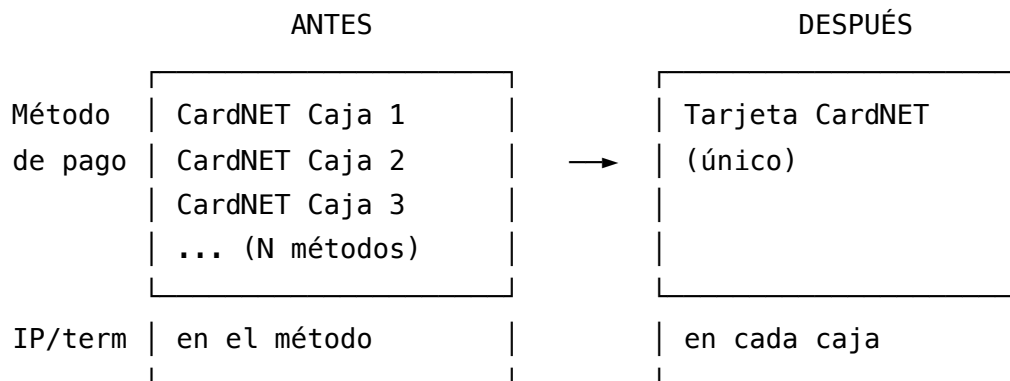


Cada línea solo lleva los campos del terminal que le corresponde. La caja arrastra una línea por cada método/terminal que use.

Flujo de pago propuesto



Antes vs Después



4. Reparto de campos: método vs línea de terminal

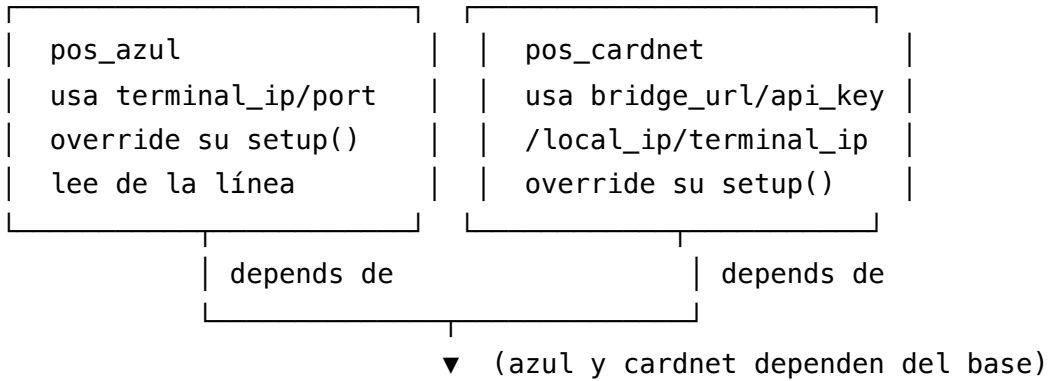
La caja puede tener **varios métodos a la vez**. Los datos de **conexión** viven en la línea del base (`pos.config.payment.terminal` , campos genéricos); los de **comportamiento** quedan globales en el método. Columna "Campo base" = nombre unificado en el módulo base.

Campo hoy (en método)	Usado por	Propuesta	Campo base (línea)
ingenico_terminal_ip	azul	línea	terminal_ip
ingenico_terminal_port	azul	línea	terminal_port
cardnet_remote_ip (Pin Pad)	cardnet	línea	terminal_ip
cardnet_remote_port	cardnet	línea	terminal_port
cardnet_bridge_url	cardnet	línea	bridge_url
cardnet_api_key	cardnet	línea	api_key
cardnet_local_ip (PC)	cardnet	línea	local_ip
ingenico_timeout / cardnet_timeout	ambos	método	— (comportamiento)
cardnet_print_mode	cardnet	método	— (comportamiento)
cardnet_enable_installments	cardnet	método	— (comportamiento)
cardnet_auto_close_batch	cardnet	método	— (comportamiento)

terminal_ip / terminal_port son comunes (IP del Ingenico o del Pin Pad).
 bridge_url / api_key / local_ip solo los usa CardNET; en líneas de Ingenico quedan vacíos.

5. Estructura de módulos: un módulo base compartido

Hoy pos_azul y pos_cardnet son **independientes** (pos_azul depende de pos_iot ; pos_cardnet de point_of_sale) y no comparten código. Se crea un **módulo base nuevo** que **contiene los campos de conexión a las terminales**. pos_azul y pos_cardnet pasan a **depender** de él y consumen esos campos.



pos_payment_terminal_base (LGPL-3) depends = [point_of_sale]	← MÓDULO NUEVO
Modelo pos.config.payment.terminal: config_id, payment_method_id + CAMPOS DE CONEXIÓN (para hablar c/ terminal): terminal_ip, terminal_port bridge_url, api_key, local_ip (CardNET) • Vista one2many en pos.config • Loader: expone las líneas al POS • Helper JS: línea por (caja, método) • Override setup() base + fallback al método	

- El **base posee todos los campos de conexión** que las terminales necesitan; los integradores **no definen campos nuevos en la línea**, solo los leen.
- Cada integrador solo aporta su override de `setup()` (cómo arma la conexión con esos campos: Ingenico HTTPS directo / CardNET vía Bridge).
- Licencia del base: **LGPL-3** (o más permisiva) para que `pos_cardnet` (proprietary) pueda depender sin conflicto.
- Instalar un integrador arrastra el base (dependencia); el modelo, los campos, la vista y el helper vienen de ahí.

6. Alcance de la solución (desarrollo personalizado)

Módulo base `pos_payment_terminal_base` (contiene los campos)

1. **Modelo `pos.config.payment_terminal`** : `config_id` (caja), `payment_method_id` + **todos los campos de conexión**: `terminal_ip` , `terminal_port` , `bridge_url` , `api_key` , `local_ip` . Restricción única (`config_id`, `payment_method_id`) si se limita a 1 línea por método (o sin restricción para multi-terminal).
2. **Vista one2many en `pos.config`** : sección "Terminales de pago" con una línea por método; los campos visibles dependen del `use_payment_terminal` del método elegido (Ingenico muestra ip/port; CardNET muestra bridge/api/ip/port).
3. **Carga al POS**: exponer las líneas de la caja al frontend (`_loader_params_pos_config` / `_pos_ui_models_to_load`), indexadas por `payment_method_id` .
4. **Helper + override base de `setup()`** : función que, dado `this.payment_method.id` , devuelve la línea de la caja activa; y fallback al método si no hay línea (transición; no rompe instalaciones existentes).

Módulos integradores (`pos_azul` , `pos_cardnet`)

5. **Añadir dependencia** al base en cada manifest. No definen campos nuevos en la línea: los consumen del base.
6. **Override de `payment_ingenico.js setup()`** : armar la conexión con los campos de la línea (Ingenico: `terminal_ip` / `terminal_port` ; CardNET: `bridge_url` / `api_key` / `local_ip` / `terminal_ip` / `terminal_port`) en vez de `this.payment_method.<campo>` .

Datos

7. **Migración (opcional)**: script que cree una línea por cada (caja, método) con la IP actual del método y consolide a un método por integrador.